Web Applications II 2025-26

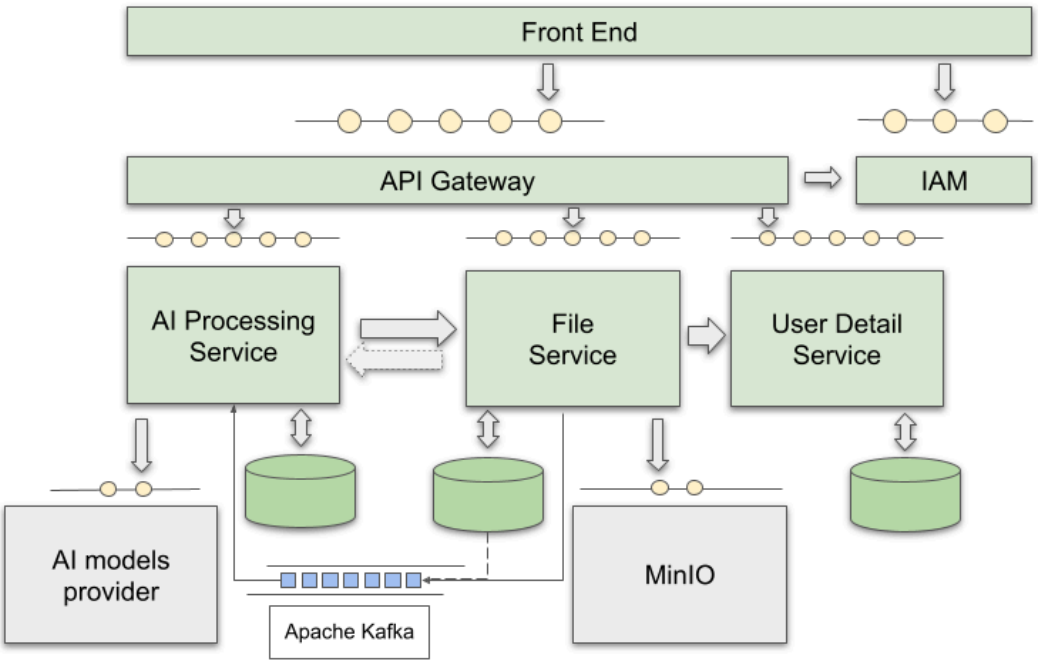
A Microservices-based file management system

Final project – A complete working system

Design and implement a **company document portal** that enables secure uploading, organizing, processing, and sharing of internal documents among authenticated users.

The system must be implemented as a **distributed**, **microservices-based** application, to ensure scalability, availability, and maintainability.
It must include a **user-friendly** front-end, which can be developed using a technology of your choice. The back-end must be implemented using *Spring Boot* in *Kotlin*. You must determine the most appropriate communication patterns between microservices (e.g., based on REST, message brokers…).

You may **extend** the system beyond the base requirements, adding **features** and **improvements** that you find useful. However, the solution must comply with the following description and final **requirements**.



Description

A *company* requires a system to **efficiently** manage internal **documents** and support **collaboration** among employees. The system must allow **secure** *uploading*, *organization*, *processing*, and *sharing* of documents within the organization.

The system is used by several kinds of users:

- **Admins**: Global system control, user and category management, full access to all documents, and monitoring of system behavior.
- **Managers**: Operate within a department, managing departmental documents and spaces.
- **Staff**: Create and manage personal documents and access departmental or explicitly shared documents.

Through a **user-friendly front-end**, users of the system should have access to the following functionalities:

1. **Document upload**
    a. Upload documents using a resumable upload mechanism
    b. Supported file types include at least PDF, DOCX, video and image files
2. **Metadata management**
    a. Associate metadata with documents, such as title, category, tags, description, and summary
    b. Manage multiple immutable versions of the same document
3. **Document access and sharing**
    a. View, download, and manage documents according to roles and permissions
    b. Share documents with other users, with explicit access rules
    c. Enforce **access control** based on roles, ownership, department, and sharing relationships
4. **Search and retrieval**
    a. Search and filter documents based on metadata and access permissions
    b. Preview documents when feasible or download them for local use
5. **Automated document processing**
    a. Execute background processing on documents after upload, such as automatic metadata generation or classification
    b. Background processing must not block user-facing operations
    c. The system must expose the status and outcome of such processing

6. **Monitoring and evaluation**
    a. Monitor the execution of background and AI-based processing pipelines
        i. Track processing status, failures, retries, and execution times
    b. Provide traceability between documents, processing events, and generated outputs
    c. Support **evaluation** and **validation** of automatically generated metadata
    d. Expose system and AI-related metrics to administrators, including **usage** and performance indicators
    e. Allow **manual intervention** on automated processing when required (e.g. re-execution or override)

General purpose monitoring and observability

Optionally, the system may offer additional **monitoring, observability, and tracing capabilities**, integrated as infrastructural components using existing tools and platforms. Their value should be reflected in **concrete system functionalities**, such as dashboards, monitoring views, or administrative insights

These extensions may include:

- **AI-specific observability and evaluation tools** (e.g. Langtrace, Langfuse or similar)
    - Track AI interactions, prompts, and responses
    - Monitor usage aspects such as token consumption, rate limits, and quotas
    - Support analysis and comparison of AI outputs over time
- **Observability and monitoring stack** (e.g. Grafana, Loki, Prometheus)
    - Collect system-wide metrics such as API usage, response times, error rates, background and AI-processing activity
    - Aggregate and store logs from multiple services to provide a unified view of system behavior
    - Provide dashboards that allow administrators to monitor system and AI pipeline health in real time, inspect failures and anomalies, and correlate system behavior with document processing activity
- **Distributed tracing system** (e.g. OpenTelemetry, tracing backends)
    - Instrument services to propagate contextual information across service boundaries
    - Enable end-to-end visibility of requests and background workflows, with particular attention to document processing and AI-related pipelines
    - Support:

- identification of latency bottlenecks
- analysis of failure points
- debugging and performance optimization through request flow visualization

Evaluation

To accept the assignment, use **GitHub Classroom** as you did during the labs:
https://classroom.github.com/a/YWu2rEXG

Once the system is ready, please contact us to schedule a presentation.
You will present your system and discuss the technical implementation you chose.
Up to **5 points** will be awarded for this project.